

Neural Network-Based European Option Pricing under Black-Scholes SDE Dynamics

Gruppo di progetto

7 maggio 2026

Sommario

Questo progetto studia se una rete neurale feed-forward possa approssimare accuratamente la funzione di pricing di Black-Scholes per opzioni call europee. Il problema viene affrontato in un ambiente controllato: i dati sono generati sinteticamente a partire dalla formula analitica di Black-Scholes, mentre la simulazione Monte Carlo viene usata come benchmark numerico. La rete neurale viene addestrata in modo supervisionato per apprendere la mappa tra parametri contrattuali e di mercato e prezzo teorico dell'opzione. L'obiettivo non è costruire un sistema di trading reale, ma valutare il ruolo di una neural network come pricing surrogate in un contesto finanziario matematicamente noto.

Indice

1	Introduzione	2
2	Framework teorico	2
2.1	Dinamica del sottostante	2
2.2	Opzione call europea	2
2.3	Formula di Black-Scholes	3
2.4	Pricing Monte Carlo	3
3	Metodologia	3
3.1	Dataset sintetico	3
3.2	Neural network	4
3.3	Training e validazione	4
3.4	Metriche	4
4	Implementazione	4
4.1	Struttura del codice	4
4.2	Efficienza computazionale	5
4.3	Riproducibilità	5
5	Esperimenti	5
5.1	Esperimento preliminare	5
5.2	Visualizzazioni previste	5
6	Risultati preliminari	6
6.1	Confronto Monte Carlo	8
7	Conclusioni e prossimi passi	8

1 Introduzione

Il pricing di strumenti derivati e' uno dei problemi classici della finanza quantitativa. Nel caso delle opzioni europee, il modello di Black-Scholes fornisce una formula chiusa per il prezzo teorico sotto ipotesi precise sulla dinamica del sottostante, sul mercato e sulla volatilita' [1]. Questa disponibilita' di una soluzione analitica rende Black-Scholes un ambiente ideale per valutare in modo controllato l'accuratezza di metodi numerici e di modelli di machine learning.

La domanda di progetto e':

Una rete neurale feed-forward riesce ad approssimare accuratamente la funzione di pricing Black-Scholes per opzioni call europee?

Il progetto confronta tre approcci:

- formula analitica di Black-Scholes, usata come ground truth;
- simulazione Monte Carlo, usata come benchmark numerico;
- rete neurale feed-forward, usata come modello supervisionato e pricing surrogate.

L'interesse principale non e' sostituire Black-Scholes nel caso in cui la formula chiusa sia disponibile, ma verificare se una rete neurale possa apprendere una funzione di pricing nota. Questo e' un primo passo utile per comprendere l'uso di modelli neurali in contesti piu' complessi, dove formule analitiche non sono disponibili o risultano computazionalmente costose.

2 Framework teorico

2.1 Dinamica del sottostante

Nel modello di Black-Scholes, il prezzo del sottostante segue un moto browniano geometrico sotto misura risk-neutral: Questa formulazione e' standard nella modellazione continua dei mercati finanziari [4].

$$dS_t = rS_t dt + \sigma S_t dW_t, \quad (1)$$

dove S_t e' il prezzo dell'asset al tempo t , r e' il tasso risk-free, σ e' la volatilita' costante e W_t e' un moto browniano standard.

La soluzione della SDE e':

$$S_T = S_0 \exp \left[\left(r - \frac{1}{2} \sigma^2 \right) T + \sigma \sqrt{T} Z \right], \quad Z \sim \mathcal{N}(0, 1). \quad (2)$$

2.2 Opzione call europea

Una call europea con strike K e maturity T ha payoff:

$$\max(S_T - K, 0). \quad (3)$$

Sotto pricing risk-neutral, il prezzo e' il valore atteso scontato del payoff:

$$C = e^{-rT} \mathbb{E}^{\mathbb{Q}} [\max(S_T - K, 0)]. \quad (4)$$

2.3 Formula di Black-Scholes

La formula analitica per una call europea e':

$$C_{BS} = S_0 N(d_1) - K e^{-rT} N(d_2), \quad (5)$$

dove $N(\cdot)$ e' la funzione di ripartizione della normale standard e:

$$d_1 = \frac{\ln(S_0/K) + \left(r + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}, \quad (6)$$

$$d_2 = d_1 - \sigma\sqrt{T}. \quad (7)$$

Nel progetto, C_{BS} viene usato come valore target per il training e come riferimento per la valutazione.

2.4 Pricing Monte Carlo

La simulazione Monte Carlo approssima il valore atteso risk-neutral generando molte realizzazioni di S_T :

$$\hat{C}_{MC} = e^{-rT} \frac{1}{M} \sum_{i=1}^M \max(S_T^{(i)} - K, 0). \quad (8)$$

Per M sufficientemente grande, $\hat{C}_{MC}$ converge a C_{BS} . Nel progetto Monte Carlo viene usato come benchmark numerico e come confronto rispetto alla rete neurale.

3 Metodologia

3.1 Dataset sintetico

Il dataset viene generato campionando casualmente i parametri dell'opzione e del mercato. Ogni osservazione contiene:

$$x = (S_0, K, T, r, \sigma), \quad (9)$$

e il target e':

$$y = C_{BS}(S_0, K, T, r, \sigma). \quad (10)$$

I range iniziali sono:

Variabile	Descrizione	Range
S_0	prezzo iniziale del sottostante	[50, 150]
K	strike	[50, 150]
T	maturity in anni	[0.1, 2]
r	tasso risk-free	[0, 0.05]
σ	volatilita'	[0.1, 0.6]

Tabella 1: Range usati per la generazione del dataset sintetico.

La scelta di dati sintetici rende il problema controllato e permette di valutare direttamente l'errore della rete rispetto alla funzione target esatta.

3.2 Neural network

Il modello usato e' una rete neurale feed-forward per regressione. La scelta e' motivata dal fatto che reti feed-forward sufficientemente ampie sono approssimatori universali di funzioni sotto ipotesi generali [2, 3]. L'architettura iniziale e':

$$5 \rightarrow 64 \rightarrow 64 \rightarrow 32 \rightarrow 1, \quad (11)$$

con funzioni di attivazione ReLU negli strati nascosti. La rete approssima la funzione:

$$f_{\theta}(S_0, K, T, r, \sigma) \approx C_{BS}. \quad (12)$$

La loss ottimizzata e' il mean squared error:

$$\mathcal{L}(\theta) = \frac{1}{n} \sum_{i=1}^n (f_{\theta}(x_i) - y_i)^2. \quad (13)$$

3.3 Training e validazione

I dati vengono divisi in training set, validation set e test set. Gli input vengono standardizzati tramite `StandardScaler`; anche il target viene scalato durante il training per migliorare stabilita' numerica e convergenza. La rete viene addestrata con Adam, early stopping e seed fissato per garantire riproducibilita'.

3.4 Metriche

Le metriche principali sono:

- Mean Absolute Error (MAE);
- Root Mean Squared Error (RMSE);
- coefficiente di determinazione R^2 ;
- MAPE calcolato solo sui prezzi maggiori di 1, per evitare instabilita' quando il prezzo teorico e' vicino a zero.

4 Implementazione

Il progetto e' implementato in Python con una struttura modulare. Le librerie principali sono NumPy, pandas, SciPy, scikit-learn, matplotlib e PyTorch.

4.1 Struttura del codice

Listing 1: Struttura principale del progetto.

```
src/option_pricing_nn/  
    black_scholes.py    # formula analitica e payoff  
    monte_carlo.py     # simulazione Monte Carlo efficiente  
    dataset.py         # generazione e salvataggio dati sintetici  
    model.py           # rete feed-forward PyTorch  
    training.py        # training loop, scaling, early stopping  
    evaluation.py      # metriche di regressione  
    plots.py          # visualizzazioni  
    pipeline.py        # esperimento end-to-end  
scripts/  
    run_experiment.py  # entry point da linea di comando
```

4.2 Efficienza computazionale

La formula Black-Scholes e la generazione del dataset sono implementate in forma vettorializzata con NumPy. Il metodo Monte Carlo usa la soluzione esatta del moto browniano geometrico al tempo T , evitando la discretizzazione temporale quando non necessaria. Inoltre, la simulazione Monte Carlo e' divisa in batch sia sulle opzioni sia sui path simulati, in modo da controllare l'uso di memoria.

Il training della rete usa `DataLoader` di PyTorch, mini-batch gradient descent, standardizzazione degli input e scaling del target. Queste scelte rendono il codice adatto sia a esperimenti rapidi sia a run piu' estesi.

4.3 Riproducibilita'

Per rendere gli esperimenti riproducibili, il progetto fissa i random seed di NumPy, Python e PyTorch. Gli artefatti prodotti da un esperimento includono:

- dataset sintetico generato;
- checkpoint del modello;
- scaler degli input;
- scaler del target;
- metriche in formato JSON;
- grafici diagnostici.

5 Esperimenti

5.1 Esperimento preliminare

E' stato eseguito un primo esperimento intermedio per verificare che la pipeline fosse corretta e che la rete fosse effettivamente in grado di apprendere la funzione di pricing. Il comando usato e':

Listing 2: Comando per un esperimento preliminare.

```
python scripts/run_experiment.py \  
--n-samples 10000 \  
--max-epochs 40 \  
--batch-size 1024 \  
--mc-n-paths 5000 \  
--mc-evaluation-samples 128
```

Questo run non rappresenta necessariamente l'esperimento finale del progetto, ma serve come controllo iniziale della correttezza dell'implementazione. Gli esperimenti finali potranno usare piu' osservazioni, piu' epoche e un numero maggiore di path Monte Carlo.

5.2 Visualizzazioni previste

Le visualizzazioni principali sono:

- andamento della training loss e validation loss;
- confronto tra prezzo Black-Scholes e prezzo predetto dalla rete;

- distribuzione degli errori;
- errore assoluto rispetto alla moneyness S_0/K ;
- errore assoluto rispetto alla maturity T ;
- errore assoluto rispetto alla volatilità σ .

6 Risultati preliminari

Nel primo esperimento intermedio, la rete ha ottenuto le seguenti metriche sul test set:

Metrica	Valore
MAE	0.6952
RMSE	0.9114
R^2	0.9985
MAPE, prezzi > 1	10.0137%

Tabella 2: Metriche preliminari della rete neurale rispetto ai prezzi Black-Scholes.

Il valore elevato di R^2 suggerisce che la rete neurale riesce già ad approssimare molto bene la funzione di pricing Black-Scholes. Gli errori assoluti medi rimangono contenuti rispetto al range complessivo dei prezzi. Il MAPE è più delicato da interpretare, perché i prezzi di opzioni molto out-of-the-money possono essere vicini a zero; per questo viene calcolato solo su prezzi maggiori di 1.

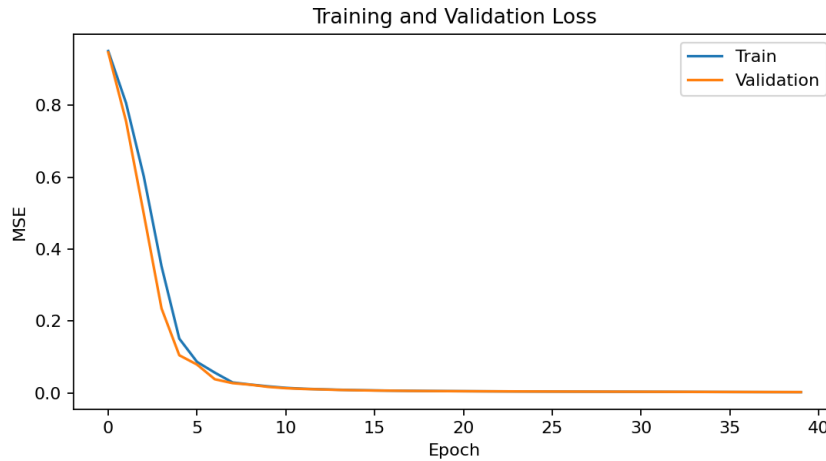


Figura 1: Training loss e validation loss.



Figura 2: Prezzi Black-Scholes rispetto ai prezzi predetti dalla rete.

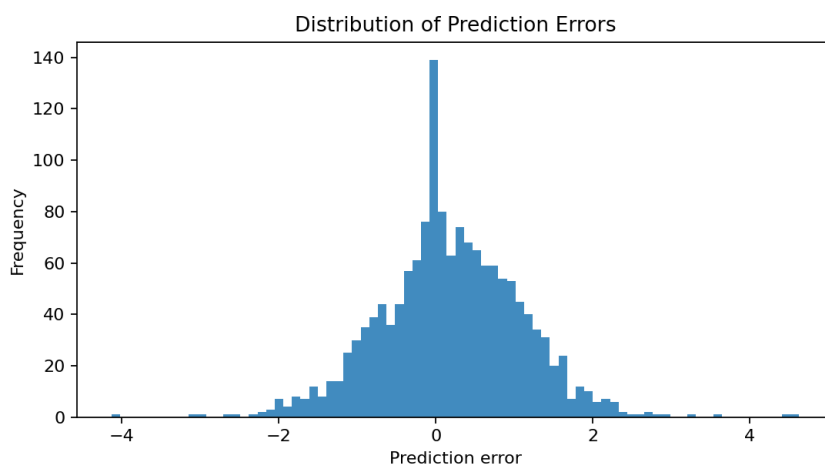


Figura 3: Distribuzione degli errori di predizione.

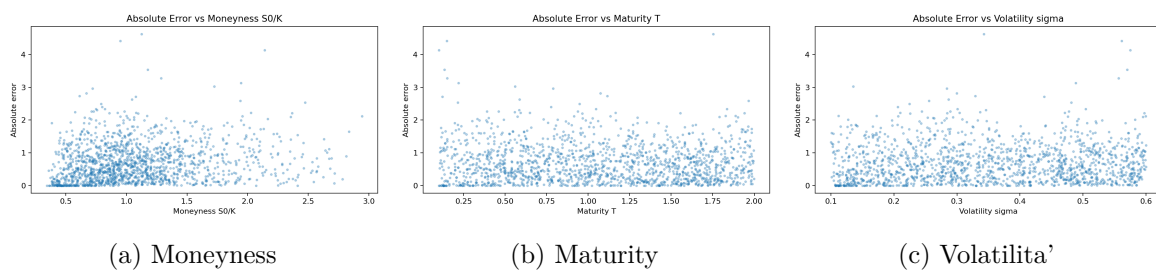


Figura 4: Errore assoluto rispetto ad alcune feature finanziarie.

6.1 Confronto Monte Carlo

Sul sottoinsieme usato per il benchmark Monte Carlo, la simulazione numerica ha ottenuto:

Metrica	Valore
MAE	0.3100
RMSE	0.5201
R^2	0.9996
MAPE, prezzi > 1	2.2139%

Tabella 3: Metriche preliminari Monte Carlo rispetto ai prezzi Black-Scholes.

Il confronto con Monte Carlo e' utile per mostrare la differenza tra un metodo numerico basato su simulazione e un modello supervisionato che, una volta addestrato, produce predizioni molto rapidamente.

7 Conclusioni e prossimi passi

La prima implementazione conferma che una rete neurale feed-forward puo' approssimare con elevata accuratezza la funzione di pricing Black-Scholes per call europee in un ambiente sintetico controllato. Il progetto e' quindi coerente con la domanda iniziale e fornisce una base solida per ulteriori esperimenti.

I prossimi passi sono:

- aumentare la dimensione del dataset sintetico;
- eseguire training piu' lunghi e confrontare diverse architetture;
- studiare l'errore per regioni specifiche dello spazio dei parametri, per esempio opzioni deep in-the-money o out-of-the-money;
- confrontare tempi di valutazione tra formula analitica, Monte Carlo e rete neurale;
- migliorare la discussione teorica sui limiti di un pricing surrogate addestrato su dati sintetici.

Come sviluppi futuri, si potrebbero considerare modelli con volatilita' stocastica, opzioni piu' complesse o metodi neurali piu' avanzati. Queste estensioni non fanno parte della prima versione del progetto, ma rappresentano direzioni naturali per generalizzare l'approccio.

Riferimenti bibliografici

- [1] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [2] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2:303–314, 1989.
- [3] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- [4] Steven E. Shreve. *Stochastic Calculus for Finance II: Continuous-Time Models*. Springer, 2004.